

Yusen Su

☎ (519)-729-1544
✉ yusen.su@uwaterloo.ca
🌐 yusen-su.github.io
👤 LinerSu
in yusu07
Nationality: Chinese (PRC)

Profile

Program-analysis researcher and engineer specializing in LLVM-based static analysis, formal verification, symbolic execution, and bounded model checking. I design and ship analysis tooling that finds real security defects in production C/C++ code (AWS C SDK, OpenHarmony OS) and contribute to open-source verification frameworks (**SeaHorn**, **Crab**).

Education

- 2020–2026 **Ph.D., Electrical & Computer Engineering**, *University of Waterloo*,
(Defended) Adviser: Arie Gurfinkel
Dissertation: *Scalable Program Analysis: Abstract Interpretation Techniques and Practical Applications*
- 2018–2020 **M.Sc., Computer Science**, *New York University (Courant Institute)*,
Adviser: Thomas Wies
- 2012–2016 **B.Sc., Computer Science**, *Univ. of Minnesota & Univ. of Electronic Science & Technology of China*

Experience

- Sep 2023 – **Associate Researcher (Part-time)**, *Huawei Technologies Canada*, Waterloo, Canada
Apr 2025
 - Built a symbolic-execution-guided test generator for C/C++ APIs: lifted targets to ELF binaries, used **angr** to reach uncovered CFG branches, and solved **Z3** path constraints to synthesize inputs for **RapidCheck** property tests
 - Raised branch coverage 40%→80% (~200 tests) on a ~1K-LOC OpenHarmony OS utility library and ~10K-LOC Huawei internal libraries; deployed in the team's CI pipeline
- May 2023 – **Associate Researcher (Intern)**, *Huawei Technologies Canada*, Waterloo, Canada
Aug 2023
 - Extended **SeaBMC** with C standard-library (string) models and a divide-and-conquer strategy—proving API-level properties and reusing them as assumptions—to verify memory safety of industrial C codebases
 - Across ~40 unit proofs, found 4 memory leaks and 5 buffer overflows in networking components, all missed by existing unit tests
- Sep 2017 **Web Developer Intern**, *Harbin Institute of Technology*, Harbin, China
Ground Station TM/TC System (**Mun**)
 - Independently designed a TCP–WebSocket proxy client and built the **Tornado** web server, including architecture and integration testing

Technical Skills

- Languages C/C++, Python, OCaml, Rust, Java, Scheme
- Compilers / LLVM (IR, passes, transformations), Git, Linux/Unix
- Systems
- Tools **Z3** (API, tactic tuning), **SeaHorn** (**SeaDSA**, **Clam/Crab**, **SeaBMC**), **CBMC**, **angr**, **KLEE**, **RapidCheck**
- Methods Abstract Interpretation, Static Analysis, Symbolic Execution, Bounded Model Checking, Taint Analysis
- Spoken Mandarin (native), English (fluent), Spanish (beginner)

Projects

2021–Present **Crab – Abstract Interpretation Library for LLVM**

- Designed and shipped 4 abstract domains and a heap-aware taint engine into **Crab**, an open-source LLVM abstract-interpretation library
- **Template DBM** — new numerical domain for buffer-bounds validation: raised the provable safe-access rate from 36% to 85% over the standard lightweight analysis (Zones), matching the most precise analysis (Convex Polyhedra) at $\sim 6\times$ its speed; on AWS C SDK + Firedancer — *VSTTE 2025, Best Tool Paper*
- **MRUD** — fast memory pre-analysis run before the heavyweight verifier (**SeaBMC**): discharges easy checks so it proves more (9 cases at 100%) and times out less on AWS C SDK code, at negligible cost ($< 2s$) — *VMCAI 2025*
- **Field-sensitive taint analysis** — tracks untrusted data through struct fields and heap objects to flag security flaws; built on **SeaDSA** + **Crab**, evaluated on Amazon ION C, CURL, and GNU Coreutils — *submitted to SAS 2026*

2025–Present **SeaDSA – Points-to Analysis for LLVM**

- Improved the analysis's field-sensitivity; plugged in the taint pipeline it cut false-positive security warnings by 4.7–13.2% across Amazon ION C, CURL, and Coreutils at only $\sim 3\%$ overhead — *submitted to SAS 2026*

2020–Present **SeaBMC – Bounded Model Checker for LLVM**

- Integrated **Crab** as a BMC pre-analysis to discharge provable assertions up front, shrinking the SMT solver's workload

2019–2020 **DRIFT² – Data Flow Refinement Type Inference**

- Implemented and evaluated a refinement-type inference engine built on abstract interpretation, showing precision and efficiency gains over Liquid Types-based verifiers — *POPL 2021*

Publications

Submitted **[SAS'26]** Y. Su, J. A. Navas, A. Gurfinkel. **Taint Analysis via Heap-Aware Propagation.**

2025 **[VSTTE'25]** Y. Su, J. A. Navas, A. Gurfinkel. **Template DBM: A New Weakly Relational Domain for Efficient Memory-Access Validation** *17th International Conference on Verified Software: Theories, Tools, and Experiments.*  *Best Tool Paper Award*

2025 **[VMCAI'25]** Y. Su, J. A. Navas, A. Gurfinkel, I. Garcia-Contreras. **Automatic Inference of Relational Object Invariants.** *Verification, Model Checking, and Abstract Interpretation.* 

2022 **[FMCAD'22]** S. Priya, Y. Su, Y. Bao, X. Zhou, Y. Vizel, A. Gurfinkel. **Bounded Model Checking for LLVM.** *22nd Formal Methods in Computer-Aided Design.* 

2021 **[ATVA'21]** S. Priya, X. Zhou, Y. Su, Y. Vizel, Y. Bao, A. Gurfinkel. **Verifying Verified Code.** *Automated Technology for Verification and Analysis.* 

2021 **[POPL'21]** Z. Pavlinovic, Y. Su, T. Wies. **Data flow refinement type inference.** *Proc. ACM Program. Lang.* 

Honors

2025 Best Tool Paper Award, VSTTE 2025

2020–present Graduate Research Studentship, University of Waterloo

Mentorship

Mentored Daniel (Xiang) Zhou, M.A.Sc. student, University of Waterloo